

Docker Compose

Multi-Container Orchestration Made Simple

From Fundamentals to Real-World Application

Prerequisites: Dockerfile | Volumes | Docker Networks

SCIT, BNU- 2026

Agenda

- 1 What is Docker Compose & Why?
- 2 Installation & CLI Basics
- 3 The Compose File In-Depth
- 4 Services, Networks & Volumes in Compose
- 5 Environment Variables & Configuration
- 6 Health Checks & Dependency Management
- 7 Scaling & Production Considerations
- 8 DEMO: Invoice Processing System
- 9 Hands-On Walkthrough & Q&A

Quick Recap

What you already know

Dockerfile

Build instructions for a single image.
FROM, RUN, COPY, CMD.
One Dockerfile = one image.

Volumes

Persist data beyond container lifecycle.
Bind mounts vs named volumes.
Shared data between containers.

Docker Networks

Container-to-container communication.
Bridge, host, overlay drivers.
DNS-based service discovery.

Now the question: How do you manage ALL of these together for a multi-service app?

Answer: Docker Compose
One YAML file to define, configure, and run
your entire multi-container application.

1

What is Docker Compose?

And why do we need it?

The Problem: Manual Container Management

Running a 3-service app manually:

```
# Step 1: Create a network
docker network create myapp-net

# Step 2: Start the database
docker run -d --name postgres \
  --network myapp-net \
  -v pgdata:/var/lib/postgresql/data \
  -e POSTGRES_PASSWORD=secret \
  postgres:16

# Step 3: Start the backend
docker run -d --name api \
  --network myapp-net \
  -e DATABASE_URL=postgresql://... \
  myapp-api:latest

# Step 4: Start the frontend
docker run -d --name web \
  --network myapp-net \
  -p 80:80 myapp-web:latest
```

Pain Points:

- X Long, error-prone commands
- X Must remember exact order
- X No single source of truth
- X Hard to reproduce across machines
- X Tedious to start/stop/rebuild
- X No built-in health checks
- X Scaling requires manual effort
- X Environment variables scattered

The Solution: Docker Compose

Docker Compose is a tool for defining and running multi-container applications.

Declarative

Define your entire stack in one YAML file

Reproducible

Same file works on any machine with Docker

One Command

docker compose up starts everything

Dependency Aware

Services start in the right order

Built-in Networking

Services discover each other by name

Easy Scaling

Scale any service with a single flag

Docker Compose vs Related Tools

Understanding the ecosystem

	Dockerfile	Docker Compose	Docker Swarm / K8s
Scope	Single image	Multi-container (one host)	Multi-host cluster
File	Dockerfile	docker-compose.yml	Stack files / manifests
Purpose	Build instructions	Dev/test orchestration	Production orchestration
Networking	N/A	Built-in bridge	Overlay networks
Scaling	N/A	Simple (--scale)	Auto-scaling, replicas
State	Stateless build	Stateful dev env	Stateful production

Key Insight: Compose handles the development workflow; Swarm/K8s handle production deployment.

2

Installation & CLI Basics

Getting started with Docker Compose

Installation

Compose V2 is built into Docker Desktop

Docker Desktop (Windows / macOS)

Compose V2 is included by default. No extra installation.

Linux

```
# Install Docker Compose plugin
sudo apt-get update
sudo apt-get install docker-compose-plugin

# Verify
docker compose version
```

V1 vs V2

V1 (Legacy)

Standalone binary: docker-compose
Separate install required
Python-based, slower

V2 (Current)

Docker CLI plugin: docker compose
Built into Docker Engine
Go-based, much faster

Always use: docker compose (with a space)
Not: docker-compose (with a hyphen)

Essential CLI Commands

docker compose up

Start all services (add -d for detached)

docker compose up --build

Rebuild images then start

docker compose down

Stop and remove containers/networks

docker compose down -v

Also remove named volumes (reset data)

docker compose ps

List running services

docker compose logs -f

Follow logs from all services

docker compose logs -f <svc>

Follow logs for one service

docker compose exec <svc> sh

Open shell in a running container

docker compose build

Build/rebuild images without starting

docker compose restart <svc>

Restart a specific service

docker compose stop

Stop without removing containers

docker compose --scale svc=N

Run N instances of a service

3

The Compose File In-Depth

Anatomy of `docker-compose.yml`

Compose File: Basic Structure

```
# docker-compose.yml

services:      # <-- Required: define
containers
  web:
    image: nginx:1.25
    ports:
      - "80:80"
  api:
    build: ./api
    ports:
      - "5000:5000"
    environment:
      - DB_HOST=db
  db:
    image: postgres:16
    volumes:
      - pgdata:/var/lib/postgresql/data
networks:      # <-- Optional: custom
networks
  default:
    driver: bridge
volumes:       # <-- Optional: named volumes
pgdata:
```

Top-level keys

services, networks, volumes
are the 3 top-level sections

Services

Each service becomes a container.
Can use image: or build:

Configuration

ports, environment, volumes,
depends_on, networks, etc.

Networks & Volumes

Declare shared resources.
Compose creates a default
bridge network automatically.

Service Configuration Options

Common keys for each service

image

Use a pre-built image from registry

```
image: postgres:16-alpine
```

build

Build from a Dockerfile

```
build: ./my_service
```

ports

Map host:container ports

```
ports: ["8080:80"]
```

environment

Set environment variables

```
environment:
```

volumes

Mount volumes or bind mounts

```
volumes:
```

depends_on

Set startup dependencies

```
depends_on:
```

networks

Attach to specific networks

```
networks: [frontend, backend]
```

command

Override container CMD

```
command: ["gunicorn", "app:app"]
```

restart

Restart policy

```
restart: unless-stopped
```

container_name

Custom container name

```
container_name: my-postgres
```

healthcheck

Define health check

```
healthcheck:
```

deploy

Resource limits (Swarm/K8s)

```
deploy:
```

4

Services, Networks & Volumes

How Compose manages Docker primitives

Networking in Compose

Automatic DNS-based service discovery

Compose automatically creates a default bridge network for all services

```
services:
  api:
    build: ./api
    networks: [frontend, backend]
  db:
    image: postgres:16
    networks: [backend]    # isolated!
  nginx:
    image: nginx
    networks: [frontend]
networks:
  frontend:
    driver: bridge
  backend:
    driver: bridge
```

Network Isolation

frontend network

Nginx

API

backend network

API

Database

Nginx cannot reach Database directly!

Service Discovery via DNS

Containers reference each other by service name

```
# In your Python code (api service)
import requests
# 'invoice-service' is the service name
# from docker-compose.yml – Docker resolves
# it to the container's IP automatically!

response = requests.get(
    "http://invoice-service:5000/invoices"
)
```

```
# In nginx.conf
upstream api-gateway {
    server api-gateway:5000;
}

location /api/ {
    proxy_pass http://api-gateway;
}
```

How It Works

- 1 Compose creates an embedded DNS server
- 2 Each service name is registered as a hostname
- 3 Containers resolve names to container IPs
- 4 No hardcoded IPs needed anywhere
- 5 Works across custom networks automatically

Important: Use hyphens, not underscores!
invoice-service (correct)
invoice_service (may cause issues with some frameworks)

Volumes in Compose

Persistent storage and data sharing

```
services:
  postgres:
    image: postgres:16
    volumes:
      - postgres_data:/var/lib/postgresql/data
      # ^named vol    ^mount point in container
  redis:
    image: redis:7
    volumes:
      - redis_data:/data
  pdf-generator:
    build: ./pdf_generator
    volumes:
      - pdf_storage:/app/pdfs # shared volume
      - ./templates:/app/templates # bind mount
volumes:
  # declare named volumes
  postgres_data: # managed by Docker
  redis_data:
  pdf_storage: # shared between services
```

Named Volumes

Managed by Docker. Persist across restarts.
Survive 'docker compose down'.
Deleted only with 'down -v'.

Bind Mounts

Map a host directory into a container.
Great for development (live reload).
Format: ./host/path:/container/path

Shared Volumes

Multiple services can mount the same volume.
Useful for file-based communication.
Example: worker writes PDF, nginx serves it.

5

Environment Variables

Configuration management in Compose

Environment Variables in Compose

Three approaches

1. Inline in YAML

```
services:
  api:
    environment:
      - DB_HOST=postgres
      - DB_PORT=5432
      - DEBUG=true
```

2. Using .env File

```
# .env file
DB_HOST=postgres
DB_PORT=5432

# docker-compose.yml
services:
  api:
    env_file: .env
```

3. Variable Substitution

```
# .env
POSTGRES_TAG=16

# docker-compose.yml
services:
  db:
    image:
      postgres:${POSTGRES_TAG}
```

Best Practice: Centralized .env File

```
# .env (shared by all services)
POSTGRES_USER=invoice_user      # Database credentials
POSTGRES_PASSWORD=secure_pass   # Never commit real secrets!
POSTGRES_DB=invoice_db
DATABASE_URL=postgresql://invoice_user:secure_pass@postgres:5432/invoice_db
REDIS_URL=redis://redis:6379/0
CELERY_BROKER_URL=redis://redis:6379/0
```

6

Health Checks & Dependencies

Ensuring services start in the right order

Health Checks

Verify that a service is truly ready

```
services:
  postgres:
    image: postgres:16-alpine
    healthcheck:
      test: ["CMD-SHELL",
            "pg_isready -U invoice_user -d
invoice_db"]
      interval: 5s      # check every 5 seconds
      timeout: 5s       # max wait per check
      retries: 5         # fail after 5 failures
      start_period: 10s # grace period on
startup

  redis:
    image: redis:7-alpine
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 3s
      retries: 5
```

Container States

starting

Container just launched, health check not run yet

healthy

Health check command returns exit code 0

unhealthy

Health check failed after all retries

Without health checks, Docker only knows if a container is running, not if the app inside is ready.

Dependency Management: depends_on

Control service startup order

```
services:
  worker-service:
    build: ./worker_service
    depends_on:
      redis:
        condition: service_healthy # wait for
        healthy
      postgres:
        condition: service_healthy # wait for
        healthy
      invoice-service:
        condition: service_started # just
        wait for start
      pdf-generator:
        condition: service_started
      notification-service:
        condition: service_started

  invoice-service:
    build: ./invoice_service
    depends_on:
      postgres:
        condition: service_healthy
      redis:
        condition: service_healthy
```

Condition Types

service_started

Waits only for the container to start.
Does NOT guarantee the app is ready.
Use for services without health checks.

service_healthy

Waits for the health check to pass.
Ensures the service is truly ready.
Requires a healthcheck: definition.

service_completed_successfully

Waits for the container to finish (exit 0).
Useful for init/migration containers.
Runs once then exits.

7

Scaling & Production Tips

Taking Compose beyond development

Scaling Services

Run multiple instances of a service

```
# Scale worker to 3 instances
docker compose up --scale worker-service=3 -d
```

```
# Check running containers
docker compose ps
```

Output:

NAME	STATUS
worker-service-1	Up 2 min
worker-service-2	Up 2 min
worker-service-3	Up 2 min
invoice-service-1	Up 5 min
postgres	Up 5 min (healthy)
redis	Up 5 min (healthy)

Scaling Considerations

No container_name

Can't scale services with fixed names

No host ports

Can't bind same host port multiple times

Load Balancing

Docker round-robins DNS queries across scaled instances

Stateless Design

Workers should be stateless for horizontal scaling

Shared Resources

All instances share the same volumes and networks

Production Tips

Best practices for real-world usage

Always pin image tags

Use postgres:16-alpine, never postgres:latest
Reproducible builds across environments

Use .env for secrets

Never hardcode passwords in docker-compose.yml
Add .env to .gitignore!

Set resource limits

Prevent any service from consuming all RAM/CPU
Use `deploy.resources.limits`

Add restart policies

`restart: unless-stopped` for production
Containers auto-recover from crashes

Use health checks

Critical for `depends_on` to work correctly
Monitoring tools rely on health status

Separate dev/prod configs

`docker compose -f base.yml -f prod.yml up`
Override files for environment-specific settings



8

DEMO: Invoice Processing System

A real-world multi-service Docker Compose application

Invoice Processing System: Overview

Real-world automation with 8 microservices

A complete invoice processing pipeline that accepts invoices, validates data, generates PDFs, sends notifications — all automatically.

Nginx

Reverse proxy
Static files
Rate limiting

nginx:1.25

API Gateway

Request routing
Health aggregation
Proxy layer

Flask + Gunicorn

Invoice Service

Invoice CRUD
Task dispatch
Business logic

Flask + SQLAlchemy

Worker Service

Async pipeline
Orchestration
Multi-step tasks

Celery

PDF Generator

HTML to PDF
Invoice templates
File storage

Flask + WeasyPrint

Notification Svc

Email simulation
Audit logging
Notification history

Flask + SQLAlchemy

PostgreSQL

Primary database
Invoice storage
Notification logs

PostgreSQL 16

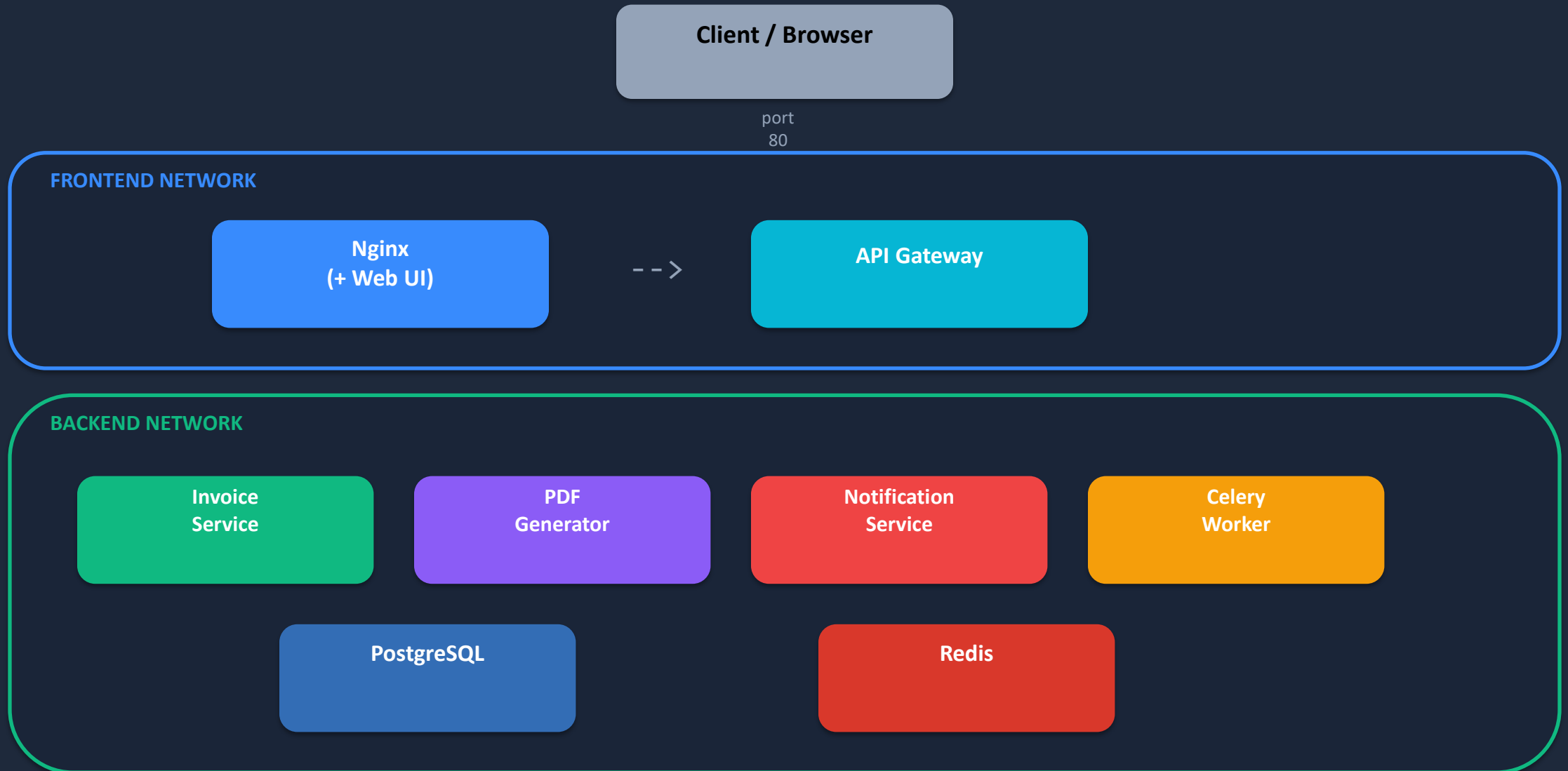
Redis

Message broker
Result backend
Task queue

Redis 7

Architecture Diagram

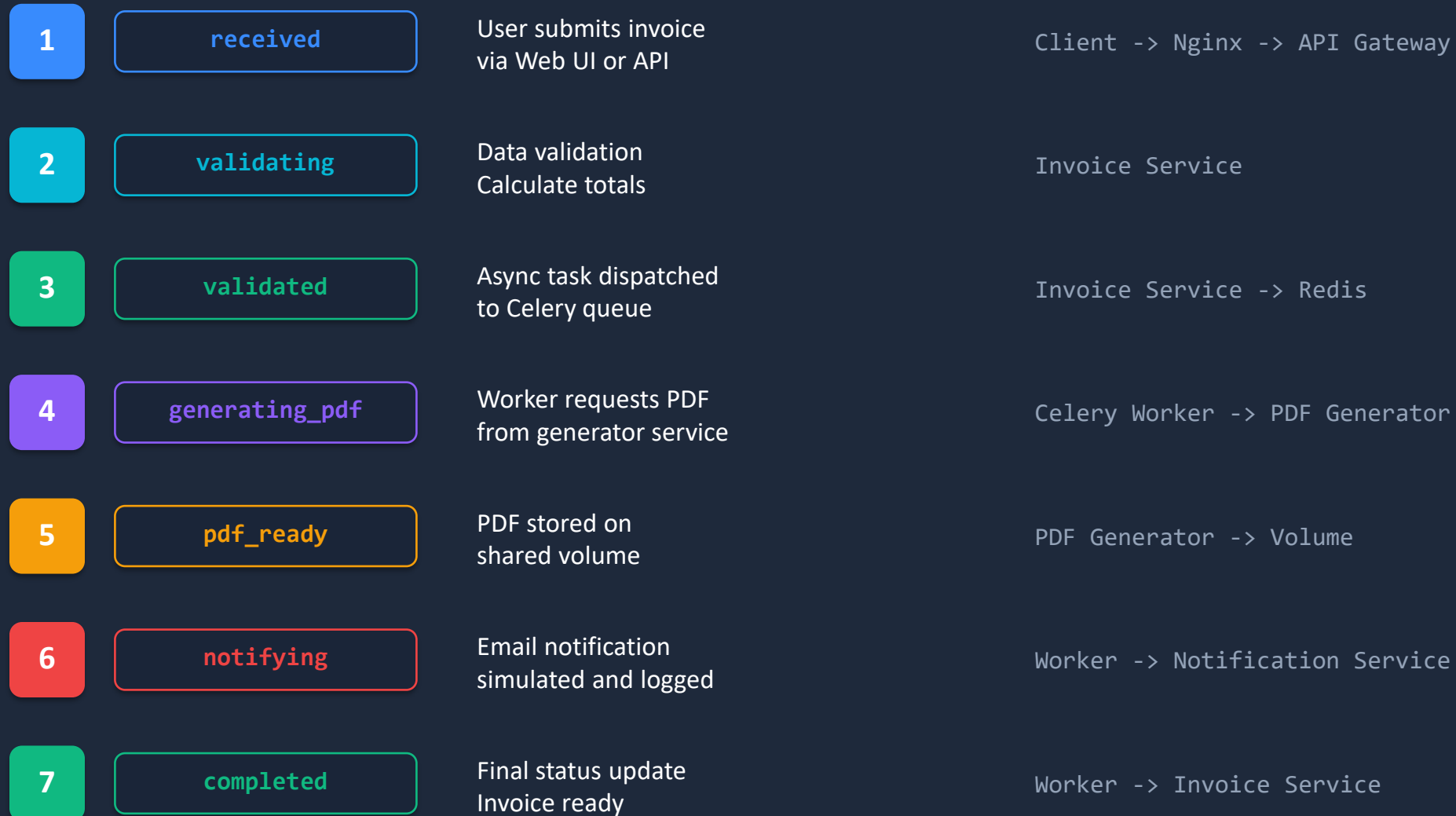
Network isolation and service communication



API Gateway spans both networks | Backend services are isolated from direct browser access | Data stores are only on backend

Invoice Processing Pipeline

7-step automated workflow



docker-compose.yml Walkthrough (Part 1)

Infrastructure services

```
services:
  postgres:
    image: postgres:16-alpine
    container_name: invoice-postgres
    environment:
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: ${POSTGRES_DB}
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./db/init.sql:/docker-entrypoint-initdb.d/init.sql
    healthcheck:
      test: ["CMD-SHELL",
        "pg_isready -U ${POSTGRES_USER} -d
        ${POSTGRES_DB}"]
      interval: 5s
      timeout: 5s
      retries: 5
    networks: [backend]
  redis:
    image: redis:7-alpine
    container_name: invoice-redis
    volumes:
      - redis_data:/data
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 3s
      retries: 5
    networks: [backend]
```

Official Images

Using pinned alpine variants
for smaller image size

Variable Substitution

All credentials come from .env
Not hardcoded!

Init Script

init.sql runs on first start
Creates tables automatically

Health Checks

pg_isready / redis-cli ping
Used by depends_on conditions

docker-compose.yml Walkthrough (Part 2)

Application services

```
        invoice-service:
build: ./invoice_service
depends_on:
  postgres: { condition: service_healthy }
  redis: { condition: service_healthy }
environment:
  - DATABASE_URL=${DATABASE_URL}
  - CELERY_BROKER_URL=${CELERY_BROKER_URL}
networks: [backend]
worker-service:
build: ./worker_service
depends_on:
  redis: { condition: service_healthy }
  postgres: { condition: service_healthy }
  invoice-service: { condition: service_started }
  pdf-generator: { condition: service_started }
  notification-service: { condition: service_started }
networks: [backend]
api-gateway:
build: ./api_gateway
networks: [frontend, backend]
nginx:
build: ./nginx
ports: ["80:80"]
depends_on: [api-gateway]
networks: [frontend]
```

Build from Dockerfile

Each service directory has its own Dockerfile + requirements.txt

Dependency Chain

Worker waits for ALL services it needs to communicate with

Multi-Network Gateway

API Gateway bridges frontend and backend networks

Single Entry Point

Only Nginx exposes port 80
All other services are internal

docker-compose.yml Walkthrough (Part 3)

Networks and volumes definition

```
networks:
  frontend:
    driver: bridge    # Nginx + API Gateway
  backend:
    driver: bridge    # All application
services
volumes:
  postgres_data:      # PostgreSQL database
  files
  redis_data:          # Redis persistence
  pdf_storage:         # Generated PDF files
```

What This Achieves:

Security

Database and internal services are not accessible from the internet

Persistence

Data survives container restarts and docker compose down

Isolation

Frontend network only contains Nginx and API Gateway

Sharing

Multiple services can access pdf_storage volume

Docker Compose Concepts Used

- Multi-service orchestration (8 services)
- Custom networks (frontend/backend isolation)
- Named volumes (3 persistent volumes)
- Health checks with conditions
- Dependency ordering (depends_on)
- Build from Dockerfile (6 custom images)
- Official images (PostgreSQL, Redis)
- Environment variables via .env
- Service scaling (--scale worker=3)
- Reverse proxy + rate limiting
- Async processing (Celery + Redis)

Web Dashboard

Modern single-page application served by Nginx

Dashboard

Live stats: total invoices, completed, processing, revenue. Recent invoices table and pipeline activity feed.

Create Invoice

Dynamic line-item form with live total calculation. Auto-tracks pipeline after submission.

Invoices List

Full table with search, status filtering, pagination. Click any row for detail modal.

Invoice Detail

Visual pipeline progress tracker with real-time polling. PDF download. Notifications viewer.

Notifications

Search and view email notification history. Filter by Invoice ID. Timestamp and delivery status.

System Status

Health check for all 8 services. Architecture diagram. API reference documentation.

Live Demo Walkthrough

Follow along with these steps

1 Build & Start

```
docker compose up --build
```

Watch the build process for each service.
Notice the dependency ordering.

2 Check Services

```
docker compose ps
```

All 8 services should be running.
Postgres and Redis show 'healthy'.

3 Open Dashboard

```
http://localhost
```

See the web dashboard with live stats.
All services show green in System Status.

4 Create Invoice

```
Use the web form
```

Add client details and line items.
Watch the pipeline progress in real-time.

5 Check Pipeline

```
Invoice Detail Modal
```

See status progression through all 7 steps.
Download the generated PDF.

6 View Logs

```
docker compose logs -f worker-service
```

Watch the async processing in real-time.
See each pipeline step execute.

7 Scale Workers

```
docker compose up --scale worker-service=3 -d
```

Create multiple invoices and see them
processed in parallel by 3 workers.

8 Clean Up

```
docker compose down -v
```

Stop all services and remove data.
Everything back to clean state.

API Reference

All endpoints accessed through Nginx on port 80

Method	Endpoint	Description
GET	/	Web Dashboard (UI)
GET	/api/health	Aggregate health check for all services
POST	/api/invoices	Create a new invoice
GET	/api/invoices	List invoices (?page=1&per_page=20&status=completed)
GET	/api/invoices/<id>	Get invoice details
GET	/api/invoices/<id>/status	Get current processing status
GET	/api/invoices/<id>/pdf	Download generated PDF
GET	/api/invoices/<id>/notifications	View notifications for invoice

Example:

```
curl -X POST http://localhost/api/invoices -H "Content-Type: application/json" -d '{"client_name":"Acme Corp","items":[{"description":"Consulting","quantity":10,"unit_price":150}]}'
```

Common Troubleshooting

Quick fixes for common issues

Port 80 in use

Change "80:80" to "8080:80" in docker-compose.yml
Access via <http://localhost:8080>

```
docker compose down && docker compose up --build
```

Database not ready

Services include retry logic.
Check postgres logs for errors.

```
docker compose logs postgres
```

Worker not processing

Verify Redis connectivity and
that all dependencies are started.

```
docker compose logs worker-service
```

PDF generation fails

WeasyPrint needs system deps.
Check the pdf-generator container.

```
docker compose logs pdf-generator
```

Stale containers

Rebuild everything from scratch
Remove all volumes and containers.

```
docker compose down -v && docker compose up --build --force-recreate
```

Host validation error

Use hyphens in service names
Not underscores (Werkzeug 3.1+)

```
Rename: invoice_service -> invoice-service
```

Key Takeaways

1 Docker Compose simplifies multi-container apps

One YAML file replaces dozens of docker run commands

2 Declarative > Imperative

Describe what you want, not how to get there

3 Networks provide security isolation

Separate frontend/backend to limit blast radius

4 Health checks enable smart dependencies

Services wait for real readiness, not just container start

5 Environment variables centralize configuration

One .env file, multiple services, no hardcoding

6 Scaling is a single flag

--scale worker=N for horizontal scalability

7 Compose is perfect for development & testing

For production, consider Docker Swarm or Kubernetes

Quick Reference

```
# Start everything
docker compose up --build -d

# Check status
docker compose ps

# View all logs
docker compose logs -f

# View one service's logs
docker compose logs -f worker-service

# Scale workers
docker compose up --scale \
  worker-service=3 -d

# Stop & clean up
docker compose down

# Full reset (data too)
docker compose down -v
```

Hands-On Exercise

Try it yourself!

1

Clone & Run

Clone the repository and run:
`docker compose up --build`
Open <http://localhost> in your browser.

2

Create an Invoice

Use the web dashboard to create an invoice.
Watch the pipeline progress in real-time.
Download the generated PDF.

3

Explore the Logs

Open a new terminal and run:
`docker compose logs -f worker-service`
Create another invoice and watch the logs.

4

Scale the Workers

Run:
`docker compose up --scale worker-service=3 -d`
Create 5 invoices rapidly.
Observe parallel processing.

5

Check Health

Run:
`docker compose ps`
Visit <http://localhost> -> System Status
See all services health check results.

6

Modify & Rebuild

Change something in a service (e.g., a log message).
Run: `docker compose up --build <service>`
See your change applied.

Further Learning & Resources

Official Documentation

docs.docker.com/compose

Complete reference for all Compose features

Compose File Reference

docs.docker.com/reference/compose-file
Every key and option explained

Docker Hub

hub.docker.com

Browse official and community images

Play with Docker

labs.play-with-docker.com

Free online Docker playground

Next Steps After This Course:

- > Docker Swarm for multi-host orchestration
- > Kubernetes (K8s) for production-grade container orchestration
- > CI/CD pipelines with GitHub Actions + Docker Compose for testing
- > Multi-stage builds for optimized production images

Questions & Discussion

Thank you for your attention!

Try the demo: `docker compose up --build` | Open `http://localhost`

Docker Compose - Multi-Container Orchestration Made Simple

Remember: Dockerfile builds one image | Compose orchestrates many containers